

Introduction to Digital Logic Design Lab

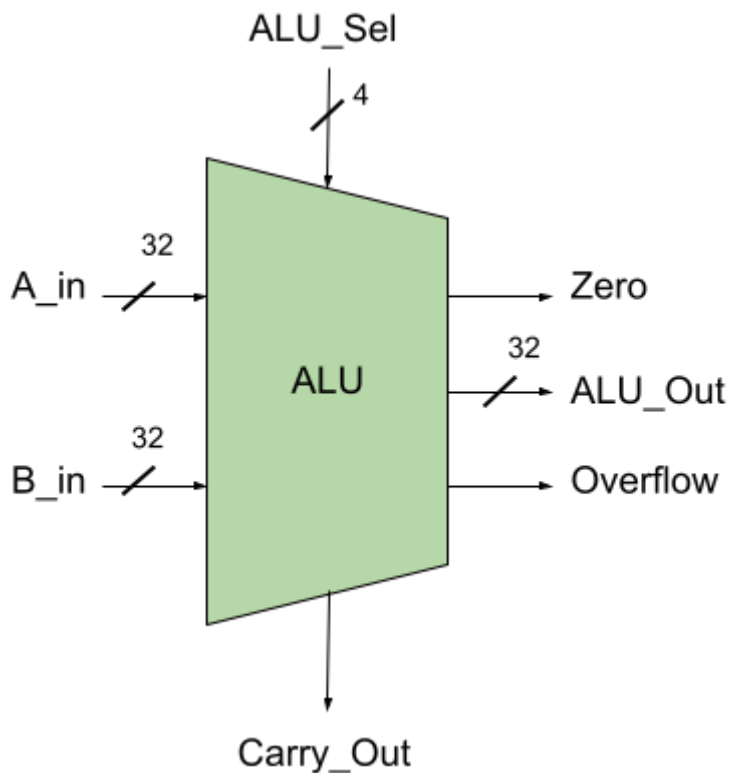
EECS 31L

Lab # 2: Arithmetic Logic Unit (ALU)

Yanilette Montano
Student ID # 81442378

2/5/2026

1 Objective



The arithmetic logic unit (ALU) performs arithmetic and bitwise operations. A_in and B_in are 32-bit inputs that will undergo the operations and the ALU_Sel is a 4-bit control input that determines which operation should be executed. Below is a control code table for the corresponding ALU operation.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	Set less than
1100	NOR
1111	Equal comparison

The 32-bit output, ALU_Out, holds the result of the arithmetic or bitwise operation the inputs undergo. The 1-bit outputs, Zero, Overflow, and Carry_Out are detection outputs depending on the operation. Zero is triggered when ALU_Out is only 0. Overflow occurs in the add and subtract operations and is triggered when two positive inputs being added result in a negative value and vice versa. Carry_Out occurs in the add operation and is triggered when a bit exceeds 32 bits or the MSB.

2 Procedure

1. Input and Output Assignment

I initialized A_in, B_in, and ALU_Sel as inputs where only A_in and B_in were 32-bits long. I initialized my outputs as Zero, Carry_Out, Overflow and ALU_Out where ALU_Out was the only 32-bit output.

2. Behavioral Logic

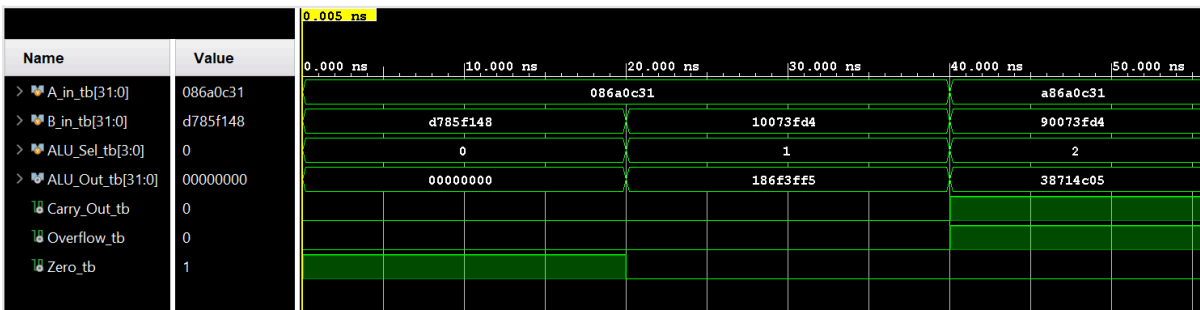
Since we had a control code table and control input, ALU_Sel, I decided to use an always block with case statements. In the sensitivity list of the always block, I included all the inputs and the expression for the case was ALU_Sel. Each case item was a 4-bit value that led to statements for an arithmetic or bitwise operation such as AND, OR,

NOR, add, subtract, set less than, and equal comparison. For my default case, I set the default operation to add and all the outputs were set to 0.

3. Flag Generation Logic

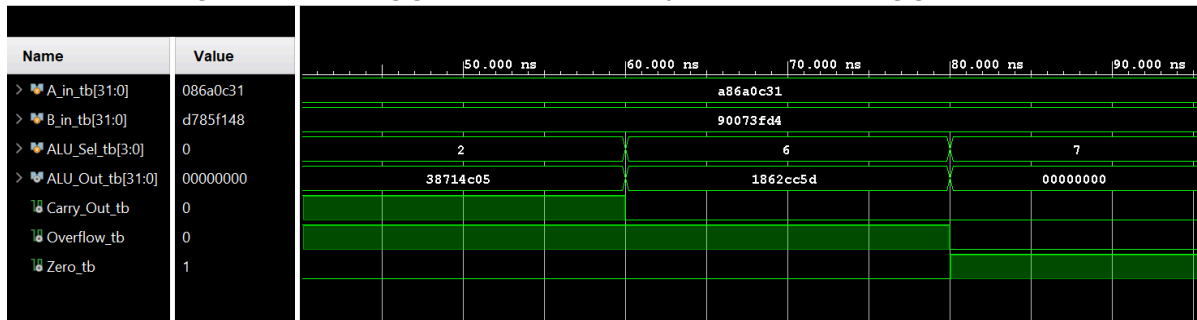
For my Zero output, I initialized it outside of the always block and in a assign statement because it's a simple logic reduction of the ALU_Out bus. For Carry_Out, I used a temporary 33-bit register variable called temp that would hold the result of addition and subtraction and set ALU_Out to the first 32-bits and turn Carry_Out on if the 33rd bit is 1. For addition Overflow, we know that it occurs when the signage of the inputs does not match the output so I used an if statement where either the inputs were negated and the ALU_Out was not, or ALU_Out was negated and the input was not, if this condition was true, overflow will turn on. For subtraction Overflow, it occurs when both the inputs have different signs. While \$signed operators were used for the arithmetic itself, distinct logical conditions were applied to the Overflow flag for addition and subtraction. This ensures that the sign-reversal is correctly identified based on the specific rules of 2's complement arithmetic for each operation.

3 Simulation Results

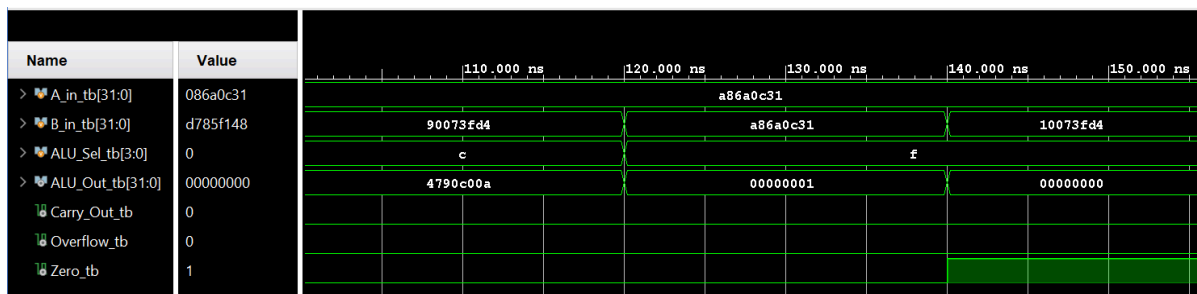


Since the A_in, B_in and ALU_Out are 32-bits, they use hexadecimal, which is a compact way to represent binary numbers. The first case is depicted in the far-left waveform. The operation executed is AND which compares each bit and ALU_Out is only triggered when each bit is one. Since our first case resulted in 0, this means that there are no bit positions where both inputs have a 1 at the same time. The Zero

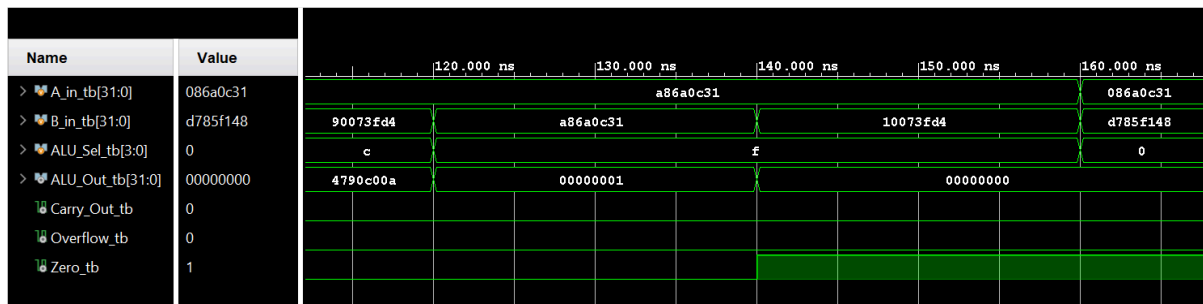
signal output is also triggered since the output result was all zeros. In the second case, the operation executed is OR, which is activated when bit positions from both inputs have a 1 or either one of them has them. In the third case, the operation executed is addition between two negative values since the MSB are both 1. The result had a 0 in the MSB which is a positive value which means overflow, in which, the Overflow signal was triggered and Carry_Out was triggered as well.



In the fourth case, the operation executed was subtraction. In signed subtraction, overflow can only happen if the signs of the input are different. In this case both inputs are the same sign which is negative values and the result is positive. This makes sense because having a very large negative number and "taking away" another large negative number (which is like adding a positive), essentially moving the value back toward zero, staying well within the 32-bit boundaries, so no overflow is triggered. In the fifth case, the operation executed is set less than where A_in is less than B_in. Since this condition does not hold true, the output is 0. The Zero signal is triggered since the output is all 0.



In the sixth case, the operation executed is NOR, in which outputs 1 when both inputs are 0. Since we performed OR with the same input values, the NOR result should be the inversion of OR. The result, 4790c00a is the inversion of b86f3fd5.



For both seventh and eighth cases, the operation executed was equal comparison where each bit is compared whether they are equal or not. For the seventh case, both inputs are identical and the output was 1. In the eighth case, the inputs were not identical and the output resulted in 0.